CO1_SentiAnalysisP2

May 25, 2025

Dela Cruz, Cloie Sofia - Morales, Rayleen Claire / AM1

IMPORT LIBRARIES

[1]:
```
# This line of code will import the pandas library into the jupyter notebook.
# Pandas is a python package that we will use to analyse and manipulate our␣
 ↪dataset.
# Using the 'as' syntax, we were able to assign the alias 'pd' to panda to␣
 ↪reference its functions easier throughout the script.

import pandas as pd
```

[2]:
```
#This line of code will import the numpy library.
#NumPy is a library in Python for the computation and manipulation of data.

import numpy as np
```

[3]:
```
#Importing train_test_split is useful for data splitting later.
#This makes dividing the training and testing data more convenient.

from sklearn.model_selection import train_test_split
```

[4]:
```
#Importing TfidVectorizer is useful for vectorizing the train and test data.
#Tokenization will also be applied while using this import.

from sklearn.feature_extraction.text import TfidfVectorizer
```

[5]:
```
#From here, it is evident what algorithm we have decided to use for training␣
 ↪our model which is Logistic Regression.
#We chose this model as it is simple and reliable for classifying given factors.
#This will later be useful in training our model for future predictions of test␣
 ↪data.

from sklearn.linear_model import LogisticRegression
```

[6]:
```
#The accuracy_score import is for showing how our model has accurately␣
 ↪predicted the sentiments of the tweets.
```

```
#The classification_report import is for detailing the performance of our model␣
 ↪through precision, recall, and its f1-score.

from sklearn.metrics import accuracy_score, classification_report
```

[7]:
```
#NLTK or Natural Language Toolkit is an import helpful for Natural Language␣
 ↪Processing (NLP), which works with human language data (NLTK, 2009).
#This library will be used for preprocessing our data for later.

import nltk
```

[8]:
```
#This import holds the list of stopwords to help filter out data for later.
#Removal of stopwords is part of the data preprocessing to easily analyze␣
 ↪significant words and ignore common, non-comprehensive data.

from nltk.corpus import stopwords
```

[9]:
```
#This import is part of the stopwords removal which splits the string into␣
 ↪words.
#This comes hand-in-hand with the filtering of stopwords and rejoining the␣
 ↪string to store into a new column as the new preprocessed data.

from nltk.tokenize import word_tokenize
```

[10]:
```
#This module holds predefined strings such as ASCII letters, digits, and even␣
 ↪whitespaces.
#For this activity, this import will be used to filter the punctuations and␣
 ↪other special characters in the data later.

import string
```

LOAD DATASET

[11]:
```
# This line of code loads our dataset, which is a CSV file named␣
 ↪'twitter_training.csv,' into a DataFrame called 'df.'
# According to W3Schools, a DataFrame is a 2D data structure which can be␣
 ↪visualized with a table with rows and columns.
# The function pd.read_csv() reads our dataset and automatically determines the␣
 ↪column headers and data types for the creation of the DataFrame df.

df = pd.read_csv('twitter_training.csv')
```

INITIAL UNDERSTANDING

[12]:
```
# The function head() displays the first five rows of the df DataFrame.
# This is a usual way for us to quickly inspect the column headers and their␣
 ↪content.
```

```
# Through this, we are able to get a quick understanding of the structure of␣
 ↪the data we are working with without loading all of it into memory.

df.head()
```

[12]:

	label	premise
0	Positive	im getting on borderlands and i will murder yo…
1	Positive	I am coming to the borders and I will kill you…
2	Positive	im getting on borderlands and i will kill you …
3	Positive	im coming on borderlands and i will murder you…
4	Positive	im getting on borderlands 2 and i will murder …

[13]:
```
# The shape() function is used to return a tuple showing the dimensionality of␣
 ↪the DataFrame df (pandas, 2024).
# The first value 74 682 is the number of records in our dataset (rows), while␣
 ↪the second value 2 is the number of columns.
df.shape
```

[13]:
```
(74682, 2)
```

[14]:
```
# The info() function from the pandas library prints more information about our␣
 ↪df DataFrame (pandas, 2024).
# It includes information like the numbers of rows (indexed from 0 to 74 681),␣
 ↪the number of columns (2), the different column names ('label' and␣
 ↪'premise'), the number of non-null rows, the data type of each columm␣
 ↪('object'), and the total memory usage of the dataset in RAM.
# It is important to note that the number of non-null values in the premise␣
 ↪column is larger than the total number of records. This means that there are␣
 ↪some values in the premise column that are null and needs to be cleaned␣
 ↪later.

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 74682 entries, 0 to 74681
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   label    74682 non-null  object
 1   premise  73996 non-null  object
dtypes: object(2)
memory usage: 1.1+ MB
```

[15]:
```
# This line of code is used to list all column names in the DataFrame df.
# Through this, we were able to confirm that expected columns like 'label' and␣
 ↪'premise' are present in the DataFrame.
```

```
df.columns
```

[15]:
```
Index(['label', 'premise'], dtype='object')
```

[16]:
```
# This prints all the unique values in the 'label' column, identifying all␣
 ↪sentiment categories in the dataset.
# For the purposes of this analysis, we have elected to delete all records with␣
 ↪'irrelevant' values because they do not contribute to our task.

print(df['label'].unique())
```

```
['Positive' 'Neutral' 'Negative' 'Irrelevant']
```

[17]:
```
# This line of code counts (with value_counts() function) and prints (with␣
 ↪print() function) the frequency of each label in the 'label' column.
# From its results, we can derive any imabalance in our sentiment␣
 ↪classification.

print(df['label'].value_counts())
```

```
label
Negative      22542
Positive      20832
Neutral       18318
Irrelevant    12990
Name: count, dtype: int64
```

[18]:
```
# This line of code filters out rows where the label is 'irrelevant.'
# We used the function str.lower() to ignore case sensitivity in the labels.
# This is an important cleaning step so that our dataset is only focused on␣
 ↪valid sentiment categories.

df = df[df['label'].str.lower() != 'irrelevant']
```

[19]:
```
# After filtering, this line of code reprints the count of each remaining␣
 ↪records of the sentiment labels.
# Through this, we can verify that all 'irrelevant' records were successfully␣
 ↪removed by the above code.

print(df['label'].value_counts())
```

```
label
Negative    22542
Positive    20832
Neutral     18318
Name: count, dtype: int64
```

[20]:
```
# The reset_index() function resets the DataFrame's index so that the row␣
 ↪numbers are sequential after filtering.
# The 'drop=True' parameter prevents the old index from being added as a new␣
 ↪column.
# On the other hand, the 'inplace=True' applied the changes directly to the df␣
 ↪DataFrame.

df.reset_index(drop=True, inplace=True)
```

[21]:
```
# The shape function will show the number of rows and columns in the df␣
 ↪DataFrame.
# The number of rows confirms that the 'irrelevant' records were filtered out␣
 ↪and that the dataset maintains its two columns.

df.shape
```

[21]:
```
(61692, 2)
```

[22]:
```
# After running the info() function again, we noticed the large number of null␣
 ↪values in the premise column.
# Because of this, we have elected to filter out these records from the df␣
 ↪DataFrame.

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 61692 entries, 0 to 61691
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   label    61692 non-null  object
 1   premise  61121 non-null  object
dtypes: object(2)
memory usage: 964.1+ KB
```

[23]:
```
# This line of code displays the records with null values in the premise column.
# The 'df['premise']' part specifies the relevant column, while the 'isnull()'␣
 ↪function specifies that only null values are to be displayed.

df[df['premise'].isnull()]
```

[23]:

	label	premise
61	Neutral	NaN
493	Neutral	NaN
511	Neutral	NaN
661	Positive	NaN
985	Positive	NaN

```
…          …      …
60994   Positive     NaN
60995   Positive     NaN
61431   Positive     NaN
61432   Positive     NaN
61433   Positive     NaN

[571 rows x 2 columns]
```

[24]:
```
# The function dropna() removes all rows from the df DataFrame where the value␣
 ↪in the 'premise' column is 'NaN.'
# The subset parameter ensure that only the 'premise' columns is checked for␣
 ↪any null values and the other column 'label' is not checked for null values.
# This line also assigns the results back to the df DataFrame to filter out and␣
 ↪records with missing text data.

df = df.dropna(subset=['premise'])
```

[25]:
```
# Similar to its usage above, the reset_index() function resets the indeces in␣
 ↪the df DataFrame.
# The parameters 'drop' and 'inplace' also makes sure to avoid adding the old␣
 ↪index as a new column in the DataFrame and applies the changes to the␣
 ↪DataFrame itself.

df.reset_index(drop=True, inplace=True)
```

[26]:
```
# This final line of code provides a short summary of the different features of␣
 ↪our df DataFrame.
# There are now less entries because we cleaned our dataset from null values␣
 ↪and 'irrelevant' records.

df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 61121 entries, 0 to 61120
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   label    61121 non-null  object
 1   premise  61121 non-null  object
dtypes: object(2)
memory usage: 955.1+ KB
```

DATA PREPROCESSING

[27]:
```
#This line of code converts all tweets under the columns 'premise' to lowercase.
#Lowercasing unifies data, reducing the need to process the text's tone from␣
 ↪its casing (Vanga, 2024).
```

```
df['premise'] = df['premise'].str.lower()
```

[28]:
```
#This function "str.replace()" aims to remove punctuation and special␣
 ↪characters through replacing "f"[{string.punctuation}]"" with a blank string␣
 ↪("").
#The parameter "regex=True" considers the "string.punctuation" as a regular␣
 ↪expression for easier pattern matching/searching within strings (Yang, 2020).
#Removal of punctuation further reduces tone ambiguity from its characters␣
 ↪(Vanga, 2024).

df['premise'] = df['premise'].str.replace(f"[{string.punctuation}]", "",␣
 ↪regex=True)
```

[29]:
```
#The English set of stopwords from the NLTK library is stored in the variable␣
 ↪'stop_words' (Geeksforgeeks, 2024).
#This set will be used later to filter out the stopwords under the column␣
 ↪'premise'.

stop_words = set(stopwords.words('english'))
```

[30]:
```
#This code applies a function to all rows of the 'premise' column to remove␣
 ↪stopwords from each tweet.
#For each row, the tweet is split into individual words, stopwords are filtered␣
 ↪out, and remaining words are rejoined into a string (Geeksforgeeks, 2024a).
#The new cleaned string is stored in a new column 'cleaned'.

df['cleaned'] = df['premise'].apply(lambda x: ' '.join(word for word in␣
 ↪word_tokenize(x) if word.lower() not in stop_words))
```

[31]:
```
#This code simply prints the data frame to observe and compare its preprocessed␣
 ↪and cleaned data before splitting it.
#The 'cleaned' column in here is not present in the CSV file itself, but only␣
 ↪in the dataframe.

print(df)
```

```
          label                                                premise  \
0      Positive  im getting on borderlands and i will murder yo…
1      Positive  i am coming to the borders and i will kill you…
2      Positive  im getting on borderlands and i will kill you all
3      Positive  im coming on borderlands and i will murder you…
4      Positive  im getting on borderlands 2 and i will murder …
…         …                                                    …
61116  Positive  just realized that the windows partition of my…
61117  Positive  just realized that my mac window partition is …
61118  Positive  just realized the windows partition of my mac …
```

```
61119  Positive  just realized between the windows partition of…
61120  Positive  just like the windows partition of my mac is l…

                                                 cleaned
0                           im getting borderlands murder
1                                    coming borders kill
2                             im getting borderlands kill
3                            im coming borderlands murder
4                          im getting borderlands 2 murder
…                                                      …
61116  realized windows partition mac like 6 years be…
61117  realized mac window partition 6 years behind n…
61118  realized windows partition mac 6 years behind …
61119  realized windows partition mac like 6 years be…
61120  like windows partition mac like 6 years behind…

[61121 rows x 3 columns]
```

DATA SPLITTING

[32]:
```
#This code assigns the 'cleaned' column to variable 'X' to prepare for data␣
 ↪splitting later.
#This column contains the cleaned and preprocessed tweets that have gone␣
 ↪through lowercasing and punctuation and stopwords removal.

X = df['cleaned']
```

[33]:
```
#This code assigns the 'label' column to variable 'y' to prepare for data␣
 ↪splitting later.
#This column contains the sentiments for each tweet, in which the model will␣
 ↪learn and aim to predict later.

y = df['label']
```

[34]:
```
#The function train_test_split() splits the data (X and y) into train and test␣
 ↪subsets, with 80% for train (train_size=0.8) and 20% for test (test_size=0.
 ↪2).
#The random_state parameter is for data shuffling before the split and stratify␣
 ↪helps split the data in proportion to the 'label' column (Scikit Learn,␣
 ↪2018).
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪train_size=0.8, random_state=3, stratify=y)
```

FEATURE EXTRACTION

[35]:
```
#This initializes the vectorizer by creating an instance.
#The instance will be used to fit and transform the cleaned data later.
vectorizer = TfidfVectorizer()
```

[36]:
```
#The 'fit_transform' method helps learn the vocabulary and the IDF from the corpus.
#This then return the TF-IDF matrix to the variable X (Geeksforgeeks, 2024).
#TF-IDF stands for Term Frequency - Inverse Document Frequency which helps in scoring a word.
#This reveals the importance of a word by contrasting the commonality and rarity of a word (Karabiber, 2024).

X_train_trans = vectorizer.fit_transform(X_train)
```

[37]:
```
#The 'transform' method helps transform our test data in its standardized form (Khanna, 2020).
#This applies the vocabulary learned from the training data.

X_test_trans = vectorizer.transform(X_test)
```

TRAINING THE MODEL

[38]:
```
#Logistic Regression is used for our model training, which is commonly known for its simple and reliable binary classification tasks through relationship analysis of given variables (Geeksforgeeks, 2024b).
#The maximum number of iterations is set to 10,000 to find the most accurate sentiment (label) prediction from the provided tweets (premises) (Scikit Learn, 2025).

model = LogisticRegression(max_iter=10000)
```

[39]:
```
#The fit() method is the training part of the model using the training data.
#The model learns from this data to prepare itself for later predictions.

model.fit(X_train_trans, y_train)
```

[39]:
```
LogisticRegression(max_iter=10000)
```

MAKING PREDICTIONS

[40]:
```
#The predict() methods is the prediction part of the model using the test data.
#Resulting predictions is stored in y_pred for performance evaluation later.

y_pred = model.predict(X_test_trans)
```

[41]:
```
#This prints the accuracy score of the model based on its predictions (y_pred) compared with the original labels (y_test).
#The accuracy score of our model is 0.8259304703476482, which reveals an 82.59% accuracy for this dataset.

print(accuracy_score(y_test, y_pred))
```

0.8259304703476482

[42]:
```
#This classification report reveals that our model predicts best on 'negative' tweets with the highest f1-score of 0.85, 'positive' predictions are high as well, with 0.83.
#However, this leaves 0.79 for the neutral tweets, which also reveals it struggles to identify this sentiment and would need further improvements.

print(classification_report(y_test, y_pred))
```

```
              precision    recall  f1-score   support

    Negative       0.84      0.86      0.85      4472
     Neutral       0.80      0.79      0.79      3622
    Positive       0.84      0.82      0.83      4131

    accuracy                           0.83     12225
   macro avg       0.82      0.82      0.82     12225
weighted avg       0.83      0.83      0.83     12225
```

Source: Kaggle - Twitter Sentiment Analysis by Passionate-NLP

Description:

- This dataset has two columns: label and premise.

- Textual data (tweets) is in the premise column, while sentiment labels are in the label column.

- Sentiment categories include: positive, negative, and neutral.

- Initially, there was an irrelevant category however, we elected to drop all records with this label so that our data will be more specific and have better polarity.

- The dataset before preprocessing contains 61 121 rows and 2 columns.

- The dataset involved data preprocessing steps such as: lowercasing, punctuation removal, stopwords removal, and tokenization (tf-idf vectorizer).

- The dataset after preprocessing contains 61 121 rows and 3 columns, with an additional 'cleaned' column for data splitting.

- Logistic regression was used for this model with an overall accuracy score of 83%.

- The model is best in predicting negative tweets with an f1-score of 0.85.

- The model needs improvement and more representations of neutral tweets with its lowest f1-score of 0.79.

BIBLIOGRAPHY

GeeksforGeeks. (2024, July 12). How to store a TfidfVectorizer for future use in scikit-learn?. GeeksforGeeks.

Geeksforgeeks. (2024a, January 3). Removing stop words with NLTK in Python. GeeksforGeeks.

Geeksforgeeks. (2024b, May 9). Understanding Logistic Regression. GeeksforGeeks.

Karabiber, F. (2024). TF-IDF — Term Frequency-Inverse Document Frequency. LearnDataSci.

Khanna, C. (2020, August 25). What and why behind fit_transform() vs transform() in scikit-learn. Towards Data Science

NLTK. (2009). Natural Language Toolkit — NLTK 3.4.4 documentation. NLTK.

Vanga, S. (2024, February 4). Text Preprocessing in Machine Learning: Finding the Hidden Value of Clean Data. Medium.

Scikit Learn. (2018). sklearn.model_selection.train_test_split — scikit-learn 0.20.3 documentation. Scikit-Learn.

Scikit Learn. (2025). LogisticRegression. Scikit-Learn.

Yang, A. (2020, January 7). Tutorial: Python Regex (Regular Expressions) for Data Scientists. Dataquest.